

PRÁCTICA 1

GNU/Linux - Resumen general completo

Introducción a los Sistemas Operativos / Conceptos de Sistemas Operativos
Facultad de Informática - UNLP - 2026

Alcance: Resumen integrado de los PDFs de Práctica 1 subidos: TP1, explicación de P1, continuación de GNU/Linux, conceptos básicos de GNU/Linux y materiales de instalación/particionado/VirtualBox. Se eliminaron duplicados, no temas.

Regla de este resumen: Cuando el TP pide investigar algo que los PDFs no desarrollan por completo, se lo marca explícitamente como pendiente de investigación. No se agregan respuestas externas como si estuvieran en el material.

Objetivo: tener un único documento para repasar la parte práctica de GNU/Linux, desde conceptos e instalación hasta comandos, usuarios, permisos, arranque e integración de ejercicios.

Mapa de contenidos

Bloque	Qué incluye
1. Base conceptual	Sistema Operativo, UNIX, GNU, Linux, software libre, GPL, características.
2. Organización	Distribuciones, kernel, shell, filesystem, FHS, utilidades.
3. Instalación y discos	Virtualización, arquitecturas, particiones, dispositivos, udev, montaje.
4. Arranque	BIOS, MBR/MBC, GPT, UEFI, Secure Boot, GRUB, bootstrap.
5. Inicio de servicios	SysV init, runlevels, inittab, scripts RC, insserv, systemd, cgroups, fstab.
6. Administración	Usuarios, grupos, permisos, archivos, editores.
7. Terminal	Procesos, comandos, redirecciones, pipes, compresión, hardware/kernel.
8. Integración	Qué exigen los ejercicios 18-24 y cómo razonar secuencias.
9. Pendientes del TP	Ítems que la guía exige investigar y no quedan completos en los PDFs.

Importante para el parcial: La explicación de práctica indica que entran los enunciados de TP, explicaciones, material adicional de cátedra y respuestas del foro. Este documento integra los PDFs disponibles; las respuestas del foro que no estén en ellos no pueden quedar incluidas.

1. Sistema Operativo y panorama de GNU/Linux

1.1 Qué es un Sistema Operativo

El material presenta al Sistema Operativo como una parte esencial del sistema de cómputo y como intermediario entre el usuario y el hardware. Su propósito es brindar un entorno cómodo y eficiente para ejecutar programas y garantizar el funcionamiento correcto del sistema.

- Administra la memoria.
- Administra la CPU.
- Administra los dispositivos y la entrada/salida.

Idea central: El usuario y los programas no administran directamente el hardware: el Sistema Operativo organiza y controla los recursos.

1.2 GNU/Linux

GNU/Linux se presenta como un sistema operativo tipo Unix (Unix-like), libre, de código abierto y disponible en múltiples distribuciones. El material insiste en distinguir Linux, que es el kernel, de GNU/Linux, que es el sistema formado por el kernel Linux más herramientas y componentes GNU.

1.3 Características generales

Característica	Qué significa en el material
Multiusuario	Permite trabajar con distintas identidades, grupos y permisos.
Multitarea	Puede ejecutar múltiples tareas/procesos.
Multiprocesador	Está preparado para aprovechar más de un procesador/núcleo.
Portable	Se desarrolló con fuerte orientación a distintas

Característica	Qué significa en el material
	arquitecturas.
Varios shells	Dispone de diferentes intérpretes de comandos, varios programables.
Usuarios y permisos	El acceso a archivos/recursos se controla por identidad y permisos.
Todo es un archivo	El modelo de GNU/Linux representa también dispositivos mediante archivos.
Jerarquía montable	Distintos directorios pueden residir en particiones distintas.
Case sensitive	Mayúsculas y minúsculas diferencian nombres: LEAME y leame no son lo mismo.
Código abierto	Permite estudiar, auditar y modificar el código.

2. De UNIX a GNU/Linux

2.1 UNIX

Los apuntes ubican el origen de UNIX en 1969 en Bell Labs, luego del trabajo previo sobre Multics. La filosofía buscó un sistema pequeño y simple, portable, multitarea y multiusuario. Entre 1969 y 1972 ganó popularidad; en 1972 fue reescrito en C y en 1973 se impulsó su distribución a universidades, facilitando nuevas versiones y adaptaciones.

Uso del término UNIX	Significado
Familia UNIX	Sistemas que comparten criterios de diseño e interoperabilidad, aunque no necesariamente código.
UNIX histórico	Línea de sistemas derivados de la implementación original de AT&T.
Marca UNIX	Marca administrada por The Open Group para sistemas que cumplen estándares publicados.

2.2 Proyecto GNU

GNU significa "GNU No es Unix". Richard Stallman inicia el proyecto en 1983 con el objetivo de construir un sistema compatible con Unix pero libre. La idea de copyleft y la GPL buscan preservar las libertades del software. En 1985 se crea la Free Software Foundation (FSF) para brindar soporte al proyecto.

1983 Inicio del proyecto GNU
 1985 Creación de la FSF
 1990 GNU ya tiene Emacs, GCC y muchas bibliotecas/utilidades
 Falta un kernel funcional
 TRIX Primer núcleo considerado; se abandona por limitaciones
 1988 Se toma MACH como base para GNU Hurd
 1991 Linus Torvalds trabaja en el kernel Linux
 1992 GNU + kernel Linux -> GNU/Linux

2.3 Software libre, propietario y GPL

El material diferencia software libre de software simplemente gratuito. La libertad se refiere a qué puede hacer el usuario con el programa, no a que necesariamente tenga costo cero.

Software libre	Software propietario
Puede usarse con cualquier propósito.	Generalmente tiene restricciones de uso.
Puede estudiarse su funcionamiento.	Normalmente no permite estudiar/modificar el código.
Puede copiarse y redistribuirse.	No suele poder redistribuirse libremente.
Puede modificarse y mejorarse.	La modificación queda bajo control del propietario.
Es común disponer del código fuente.	Generalmente no se entrega el código fuente.

Software libre	Software propietario
Correcciones pueden surgir de la comunidad.	Correcciones dependen del propietario.

La GPL de GNU se presenta como la licencia creada para proteger la libre distribución, modificación y uso del software publicado bajo ella. La explicación práctica ubica su creación en 1989 y menciona GPLv3 como versión actual del material.

2.4 Linux

Linux es el kernel iniciado por Linus Torvalds en 1991, originalmente orientado a procesadores Intel 80386 y luego distribuido bajo GPL. El material menciona la versión 1.0 en 1994, la mascota Tux y kernel.org como sitio de referencia.

No confundir: Linux = kernel. GNU/Linux = sistema operativo que combina ese kernel con componentes GNU y otros paquetes.

3. Distribuciones

Una distribución es una customización o conjunto de software que toma el kernel Linux, herramientas GNU y otros paquetes, y los integra con configuraciones, instalador, gestor de paquetes, entorno gráfico y utilidades propias.

- Las distribuciones pueden diferenciarse por selección de paquetes, herramientas de configuración, administración de software, objetivos y preferencias de usuario.
- Una distribución puede derivar de otra.

Familia / distribución	Ejemplos que aparecen en los PDFs
Principales mencionadas	Debian, Arch, Gentoo, Red Hat Linux, Slackware, openSUSE.
Derivadas de Debian	Ubuntu; el material también menciona Mint como basada en Ubuntu.
Derivadas / continuidad Red Hat	Fedora, Red Hat Enterprise, Rocky Linux, CentOS.
Basadas en Ubuntu	Edubuntu, Kubuntu, Mint.
Basada en Slackware	SLAX.

TP1 pide investigar: El punto 2 exige al menos cuatro distribuciones, diferencias básicas y una investigación específica de Debian (objetivos y cronología desde debian.org). Los PDFs nombran Debian y familias, pero no desarrollan completa esa investigación.

4. Estructura básica de GNU/Linux

Los materiales sintetizan la estructura del sistema alrededor de tres componentes fundamentales: kernel, shell y filesystem. La explicación práctica agrega las utilidades como el conjunto de software que ayuda a diferenciar una distribución de otra.

Componente	Función
Kernel	Núcleo: administra CPU, memoria, E/S, dispositivos y la relación software-hardware.
Shell	Interfaz de línea de comandos: interpreta órdenes y ejecuta programas.
Filesystem	Organiza y administra archivos y directorios en dispositivos de almacenamiento.
Utilidades	Editores, networking, oficina, interfaces gráficas y demás software del sistema.

5. Kernel Linux

5.1 Funciones y arquitectura

El kernel ejecuta funciones esenciales en modo privilegiado, gestiona dispositivos y coordina la CPU, la memoria y la entrada/salida. El material lo describe como un núcleo monolítico híbrido: gran parte del código y los drivers ejecutan con privilegios, pero la funcionalidad puede cargarse o descargarse mediante módulos.

- Los Loadable Kernel Modules permiten incorporar o retirar drivers/extensiones con el sistema funcionando.
- El código se desarrolla mayoritariamente en C, con partes en assembler; el proceso de construcción usa además herramientas y lenguajes auxiliares.
- Linux evolucionó hacia una gran portabilidad y puede funcionar en numerosas arquitecturas.

5.2 Versionado A.B.C[.D]

Campo	Idea del apunte
A	Versión principal; cambia con poca frecuencia y ante cambios grandes.
B	Revisión mayor. Antes de 2.6, par/estable e impar/desarrollo; desde 2.6 esa distinción deja de ser central.
C	Revisión menor; incorpora drivers o características.
D	Correcciones puntuales / bug-fixes en el esquema histórico explicado.

5.3 Kernel, máquinas virtuales y licencia

El kernel puede ejecutarse en entornos virtualizados como host o como huésped. El material también destaca que Linux pasó a distribuirse bajo GNU GPL, permitiendo distribución y modificaciones bajo sus términos.

TP1 y /boot: La guía pregunta si pueden coexistir varios kernels y orienta a observar /boot. Los materiales de arranque muestran /boot/grub y archivos asociados al kernel/arranque; la respuesta completa del inciso se debe verificar en la instalación.

6. Shell e intérprete de comandos

El shell es el programa que recibe órdenes textuales, las interpreta y solicita su ejecución al sistema. No forma parte del kernel: dialoga con él. Puede existir un shell distinto por usuario y la asignación puede verse en /etc/passwd.

Shell	Características que destaca el material
sh	Bourne Shell; base histórica y muy portable.
csh	Sintaxis influida por C; tradicional en BSD.
ksh	Korn Shell; basado en sh, con historial y mejoras interactivas.
bash	Bourne Again Shell; muy usado en GNU/Linux, history, autocompletado, redirecciones.
zsh	Integra características de otras shells y agrega funciones propias.

6.1 Cómo resuelve un comando

Según el apunte, cuando el shell recibe una orden busca primero si es una orden interna; luego considera alias; después busca ejecutables externos en las ubicaciones de la variable PATH.

Orden ingresada
-> comando interno del shell?

```
-> alias?
-> búsqueda en $PATH
-> ejecutable externo
```

- Los comandos internos son ejecutados directamente por el shell.
- Los externos pueden estar en /bin, /usr/bin, /usr/local/bin u otros directorios agregados a PATH.
- Un nombre puede existir como comando interno y externo; el path completo permite forzar el externo.
- Comandos de ayuda mencionados: type, help y man.

7. Filesystem, FHS, rutas y archivos

7.1 Qué administra un filesystem

El filesystem define cómo se organizan y administran archivos en almacenamiento. El apunte incluye métodos de acceso, manejo/protección de archivos, administración del espacio en memoria secundaria y mecanismos de integridad.

- Los datos suelen organizarse en bloques/sectores.
- Los directorios asocian nombres con archivos y estructuras de control, como i-nodos en sistemas Unix.
- La estructura puede ser jerárquica en forma de árbol.

7.2 Filesystems mencionados

Los PDFs mencionan FAT, NTFS, ext2/ext3/ext4, ReiserFS, XFS y Btrfs en diferentes contextos. La explicación práctica señala a la familia Extended como habitual en GNU/Linux y presenta Btrfs como alternativa con funciones como snapshots, compresión y tolerancia a fallas.

7.3 FHS - directorios principales

Ruta	Contenido / finalidad según los apuntes
/	Raíz de toda la jerarquía.
/bin	Binarios esenciales, por ejemplo cp, ls, mv.
/dev	Archivos que representan/acceden a dispositivos.
/etc	Archivos de configuración del sistema.
/sbin	Programas esenciales de administración del sistema.
/home	Directorios personales de usuarios.
/lib	Bibliotecas compartidas esenciales.
/proc	Filesystem virtual con información de procesos, kernel, CPU, memoria, etc.
/root	Home del usuario root.
/tmp	Archivos temporales.
/usr	Gran parte de programas, bibliotecas y datos no esenciales para el arranque básico.
/usr/bin	Programas que no se encuentran en /bin.
/usr/include	Headers, por ejemplo para C.
/usr/lib	Bibliotecas y archivos asociados a programas.
/usr/local	Software/localizaciones específicas del sistema.
/usr/src	Código fuente; el apunte ejemplifica con fuentes del kernel.
/var	Datos que cambian o crecen.
/var/log	Logs e históricos del sistema.
/var/spool	Datos en espera de ser procesados por otro programa.
/boot	Contexto de arranque: GRUB e imágenes/archivos usados durante el boot en los materiales.

7.4 Rutas y nombres

Concepto	Ejemplo / significado
Path absoluto	Empieza en / y especifica la ubicación completa: /home/usuario/archivo.
Path relativo	Se interpreta desde el directorio actual: ../archivo.
.	Directorio actual.
..	Directorio padre.
~	Home del usuario actual.
\$HOME	Variable que contiene el home del usuario.
Case sensitive	LEAME y leame son nombres diferentes.

Los apuntes remarcan que GNU/Linux no depende de la extensión del nombre para saber el tipo real de archivo. El comando file permite consultar el tipo.

7.5 Enlaces

El TP pide investigar ln. El apunte explica el concepto de enlace duro como una referencia a los datos físicos y utiliza ln/ln -s dentro del repertorio de comandos. Un enlace simbólico apunta a otro nombre/ruta y se distingue en listados detallados.

8. Instalación, arquitectura y virtualización

8.1 Soportes y arquitecturas

La continuación de GNU/Linux indica que la instalación depende de la arquitectura de hardware y puede realizarse desde una imagen ISO o desde USB.

Arquitectura mencionada	Descripción del material
amd64	Arquitectura de 64 bits.
i386	Arquitectura de 32 bits.
arm / armel	Familia Advanced RISC Machine.
ia64	Intel Itanium / Intel Architecture-64.

También se menciona UNetbootin como herramienta para crear instaladores/LiveCD en USB.

8.2 Emulación, virtualización y paravirtualización

Modelo	Idea clave
Emulación	Implementa/emula hardware e instrucciones; costosa pero puede ejecutar arquitecturas distintas a la física.
Virtualización completa	Ejecuta huéspedes sobre un host mediante hypervisor; aprovecha soporte de la arquitectura y es más eficiente que emular.
Paravirtualización	Usa sistemas huéspedes modificados para entornos virtualizados; busca mayor eficiencia.

Para la práctica se utiliza VirtualBox como entorno controlado, asignando recursos de memoria, CPU y almacenamiento a la máquina virtual.

8.3 Flujo de instalación de Debian en VirtualBox

1. Instalar VirtualBox y obtener la imagen de Debian adecuada para la arquitectura.
2. Crear la máquina virtual, definir nombre y asignar RAM, CPU y disco virtual.
3. Adjuntar la ISO y configurar el arranque desde el medio de instalación.
4. Iniciar el instalador, configurar idioma, ubicación, hostname, root y usuario.
5. Realizar el particionado manual según el ejercicio/guía.
6. Instalar el sistema base y paquetes seleccionados.

7. Instalar GRUB en el disco indicado.
8. Reiniciar, probar login y consola.
9. Retirar/desmontar la ISO para arrancar desde el sistema instalado.

8.4 Layout de particiones del ejemplo de instalación

No confundir mínimo con ejemplo: La teoría dice: mínimo 1 partición para /; recomendado al menos / + swap. La guía de instalación usa un esquema más completo con /, /boot, swap y /home.

Uso	Tipo/FS según la guía de instalación
/ (raíz)	Primaria, con Ext4 por defecto en el ejemplo.
/boot	Primaria, punto de montaje /boot y flag de arranque en la guía.
swap	Partición lógica usada como área de intercambio.
/home	Partición con el espacio restante para datos de usuarios.

8.5 Ejercicio histórico de particionado

El material de laboratorio usa una VM con una imagen antigua de MS-DOS/Windows 3.11 para practicar conceptos de boot y particiones. Primero se inspecciona el disco con fdisk; luego se agrega una imagen de floppy, se arranca desde ella y se usa FIPS para redimensionar una partición y liberar espacio. Finalmente se vuelve a arrancar desde el disco y se verifica el nuevo esquema.

9. Discos, particiones y nombres de dispositivos

9.1 Particiones

Una partición divide lógicamente un disco físico. Cada partición puede destinarse a un sistema operativo o a un filesystem determinado, y separar sistema/datos puede facilitar administración, recuperación o protección. El material también advierte que particionar en exceso puede reducir flexibilidad.

Tipo	Descripción
Primaria	División registrada directamente en la tabla MBR. En el esquema clásico puede haber hasta 4.
Extendida	Una de las entradas primarias puede actuar como contenedor de particiones lógicas; no recibe directamente un filesystem en el esquema explicado.
Lógica	Reside dentro de la extendida y puede tener su propio filesystem. En la nomenclatura clásica comienza desde 5.

9.2 Herramientas de particionado

Tipo	Ejemplos del material
Destructivo	fdisk: crear/eliminar particiones.
No destructivo	FIPS, GParted: crear/eliminar/modificar o redimensionar según el material.

9.3 Nomenclatura de discos

Tecnología	Nombres del material
IDE clásico	/dev/hda master bus 1; /dev/hdb slave bus 1; /dev/hdc master bus 2; /dev/hdd slave bus 2.
SCSI	/dev/sda, /dev/sdb, /dev/sdc...
SATA	Usa la nomenclatura sdX, igual que SCSI en los apuntes.
Particiones	Primarias 1-4; lógicas desde 5 en el esquema MBR clásico.

La continuación explica que, desde Debian/Squeeze, dispositivos antes llamados hdX pasan a la nomenclatura sdX. También introduce udev como gestor dinámico de /dev y el Persistent Device Naming.

```
Nombres persistentes
/dev/disk/by-uuid/... -> ../../sda1
/dev/disk/by-label/data -> ../../sdb2
```

9.4 Del disco al archivo

DISCO -> PARTICIÓN -> FILESYSTEM -> PUNTO DE MONTAJE -> ARCHIVOS

Los comandos y archivos asociados a este flujo aparecen repetidamente: fdisk para particiones, mkfs para crear filesystem, mount/umount para montar/desmontar, df/du para uso de espacio y /etc/fstab para montajes configurados al inicio.

10. MBR, BIOS, GPT, UEFI y Secure Boot

10.1 BIOS, MBR y MBC

En el modelo clásico x86, BIOS inicia el hardware, ejecuta POST y termina leyendo el código de arranque del MBR. El MBR ocupa un sector de 512 bytes en el esquema explicado.

Parte del MBR	Tamaño / función en los PDFs
Master Boot Code (MBC)	446 bytes; código pequeño que permite continuar el arranque.
Tabla de particiones	64 bytes; contiene la información del esquema de particiones MBR.
Firma final	2 bytes al final del sector, según la explicación.

10.2 GPT

GPT (GUID Partition Table) se presenta como un esquema asociado a EFI/UEFI que supera limitaciones del MBR, especialmente en cantidad/tamaño de particiones. Usa direccionamiento LBA.

- LBA 0 conserva un MBR de compatibilidad.
- LBA 1 contiene la cabecera GPT.
- La tabla de particiones ocupa bloques sucesivos.
- Cabecera y tabla se mantienen al inicio y al final del disco para redundancia.

10.3 EFI y UEFI

EFI define una interfaz entre sistema operativo y firmware. UEFI, impulsado por UEFI Forum, moderniza el proceso de arranque y expone información/servicios para bootloaders y aplicaciones UEFI.

- Incluye Boot Manager para cargar aplicaciones/bootloaders UEFI y drivers.
- El bootloader puede residir como aplicación en un filesystem UEFI, presentado en el material con FAT32.
- UEFI agrega capacidades como interfaz gráfica, autenticación de red y criptografía.

10.4 Secure Boot

El material lo presenta como un mecanismo para validar aplicaciones y drivers UEFI antes de ejecutarlos. Usa firmas y pares de claves asimétricas; el firmware conserva claves públicas autorizadas para verificar imágenes.

11. Gestores de arranque y bootstrap

11.1 Bootloader

El bootloader carga una imagen de kernel desde una partición y le transfiere el control. Los materiales nombran GRUB, LILO, NTLDR, GAG y YaST. Puede instalarse en el MBR o en el sector de arranque de una partición (VBR) en el modelo explicado.

11.2 GRUB Legacy

1. Fase 1 en el MBR: inicia la siguiente fase.
2. Fase 1.5 en el MBR gap: permite llegar a la fase 2.
3. Fase 2: interfaz y carga del kernel seleccionado.

Se configura en `/boot/grub/menu.lst` en el esquema Legacy. El material muestra parámetros como `default`, `timeout`, `title`, `root`, `kernel` e `initrd`.

11.3 GRUB 2

GRUB 2 agrega soporte para más arquitecturas, caracteres/idiomas y personalización. La explicación destaca que ya no usa la fase 1.5 de GRUB Legacy y que el archivo pasa a `/boot/grub/grub.cfg`, que no debería editarse manualmente; se menciona `update-grub`.

11.4 Cadena general de arranque

```
ENCENDIDO
-> BIOS/UEFI
-> bootloader
-> kernel + initrd
-> PID 1 (init o systemd, según el modelo)
-> servicios/unidades
-> login / sistema disponible
```

Shutdown: El TP pide describir el proceso completo de parada. Los PDFs incluyen `runlevel 0`, `init 0` y comandos `shutdown/halt`, pero no desarrollan una secuencia de apagado con el mismo detalle de las 12 etapas de SysV init.

12. Proceso de arranque SysV init

12.1 Las 12 etapas del material

1. Comienza la ejecución del BIOS.
2. BIOS ejecuta POST.
3. BIOS lee el sector de arranque (MBR).
4. Se carga el gestor de arranque (MBC).
5. El bootloader carga kernel e `initrd` (initial ram disk).
6. Se monta `initrd` como filesystem raíz y se inicializan componentes esenciales.
7. El kernel ejecuta `init` y se desmonta `initrd`.
8. Se lee `/etc/inittab`.
9. Se ejecutan scripts apuntados por el `runlevel 1`.
10. El final del `runlevel 1` indica pasar al `runlevel` por defecto.
11. Se ejecutan scripts del `runlevel` por defecto.
12. El sistema queda listo para ser usado.

12.2 Proceso init

- Se ejecuta desde `/sbin/init` en el modelo mostrado.
- Tiene PID 1.
- No tiene padre y aparece como ancestro del resto de procesos; `pstree` permite observar la jerarquía.
- Carga/inicia subprocesos necesarios para el funcionamiento del sistema.
- En SysV init se configura mediante `/etc/inittab`.

12.3 Runlevels

Nivel	Significado estándar del material
0	halt: parada/apagado.
1	single-user: modo monousuario.
2	multi-user without network support en el estándar mostrado; el material señala Debian como ejemplo con <code>runlevel 2</code> por defecto.
3	multi-user console mode; Red Hat aparece como ejemplo

Nivel	Significado estándar del material
	de runlevel 3 por defecto.
4	No asignado / N/A en el esquema presentado.
5	X11: multiusuario con entorno gráfico.
6	reboot: reinicio.

12.4 /etc/inittab

id:runlevels:acción:proceso

Campo	Significado
id	Identifica la entrada.
runlevels	Niveles donde se aplica la acción.
acción	Cómo se ejecuta el proceso: wait, initdefault, ctrlaltdel, off, respawn, once, sysinit, boot, bootwait, powerwait, etc.
proceso	Comando exacto a ejecutar.

12.5 Scripts RC, orden e insserv

- Scripts: normalmente en /etc/init.d.
- En /etc/rcX.d hay enlaces simbólicos para cada runlevel.
- Patrón: [S|K]<orden><nombre>.
- S inicia el script (start); K lo detiene (stop).
- El número de dos dígitos determina el orden relativo.

El comando insserv administra el orden de los enlaces y resuelve dependencias a partir de cabeceras LSBinit. El material indica que los scripts deben ofrecer al menos start, stop, restart, force-reload y status, retornar códigos de salida apropiados y declarar dependencias.

```
### BEGIN INIT INFO
# Provides:          scriptname
# Required-Start:    $remote_fs $syslog
# Required-Stop:     $remote_fs $syslog
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
### END INIT INFO
```

TP1 pide investigar: El cambio de runlevel X -> Y y si es permanente aparece como consigna. Los PDFs muestran init 0/init 6 y el concepto de runlevel, pero no desarrollan en detalle la persistencia del cambio para cualquier X/Y.

13. systemd, units, socket activation, cgroups y fstab

13.1 systemd

systemd centraliza la administración de demonios/servicios y librerías del sistema, mejora el paralelismo de arranque y puede controlarse con systemctl. En el modelo moderno mostrado, systemd reemplaza a init como PID 1, los runlevels son sustituidos por targets y ya no se utiliza /etc/inittab.

Unit	Función
.service	Controla un servicio particular.
.socket	Encapsula IPC/socket/FIFO y permite activación por socket.
.target	Agrupar units o establece puntos de sincronización/dependencias.
.snapshot	Guarda el estado de un conjunto de unidades para restaurarlo más tarde.

Las units pueden encontrarse en estados active o inactive, según el material.

13.2 Activación por socket

La explicación muestra el problema de iniciar servicios que quizá nunca se utilicen. Con socket activation se inicia primero el socket y, cuando llega una conexión, systemd inicia el servicio y le entrega el socket. La idea es activar servicios bajo demanda y favorecer el paralelismo.

13.3 cgroups

Los cgroups permiten organizar jerárquicamente conjuntos de procesos relacionados. El material los vincula con seguimiento de procesos y con límites/controles sobre recursos como CPU, memoria y discos.

13.4 /etc/fstab

/etc/fstab define filesystems/particiones que pueden montarse durante el arranque. El formato de ejemplo contiene: filesystem, punto de montaje, tipo, opciones, dump y pass.

Opción	Significado indicado
user	Permite que un usuario monte la partición.
auto	Monta al inicio.
ro	Solo lectura.
rw	Lectura y escritura.

```
/dev/sda1 / ext4 errors=remount-ro 0 1
/dev/sda2 none swap sw 0 0
UUID=... /home/iso/datos ntfs user,auto,rw,... 0 2
```

14. Usuarios y grupos

Todo usuario posee credenciales e identidad en el sistema. El material distingue root, usuarios regulares y usuarios regulares habilitados para tareas de superusuario mediante /etc/sudoers. Cada usuario puede pertenecer a uno o más grupos y puede tener un home.

Archivo	Información
/etc/passwd	Información de cuentas; el ejemplo muestra nombre, campos de identidad, home y shell.
/etc/group	Grupos y membresías.
/etc/shadow	Información protegida de contraseñas/expiración.
/etc/sudoers	Usuarios/reglas para ejecutar tareas con privilegios, mencionado en la clasificación de usuarios.

14.1 Comandos de administración

Comando	Función / parámetros que aparecen
useradd usuario	Agrega un usuario; modifica información de cuentas. El material menciona adduser como alternativa.
passwd usuario	Asigna o cambia contraseña; el material lo relaciona con /etc/shadow.
usermod	-g grupo primario; -G grupos adicionales; -d directorio home.
userdel	Elimina usuario.
groupadd / addgroup	Crea grupo.
groupdel / delgroup	Elimina grupo.
su	Cambia de usuario / inicia sesión como otro usuario según

Comando	Función / parámetros que aparecen
	contexto.
who / users	Muestran usuarios conectados.
id / groups	Muestran identidad, UID/GID y grupos según el repertorio de comandos.

TP1 pide investigar: UID/GID, coexistencia de UIDs iguales, UID de root y si puede existir más de una cuenta con perfil equivalente a root son preguntas explícitas del TP; los PDFs no desarrollan completamente esas justificaciones.

15. Permisos

15.1 Modelo usuario-grupo-otros

Cada archivo o directorio tiene un usuario dueño y un grupo dueño. Los permisos se evalúan para usuario/owner (u), grupo (g) y otros (o).

Permiso	Letra	Valor octal
Lectura	r	4
Escritura	w	2
Ejecución	x	1

Número	Permisos
0	---
1	--X
2	-W-
3	-WX
4	r--
5	r-X
6	rW-
7	rWX

```
chmod 755 script -> u=rwx, g=r-x, o=r-x
chmod 640 archivo -> u=rw-, g=r--, o=---
chmod u+x archivo -> agrega ejecución al dueño
chown usuario:grupo archivo
chgrp grupo archivo
```

Los apuntes muestran -R para aplicar cambios de propiedad/grupo/permisos de forma recursiva en comandos que lo soportan.

15.2 Permisos en directorios

Permiso	Efecto sobre directorio según el apunte
r	Permite ver/listar su contenido, no implica leer el contenido de los archivos.
w	Permite añadir o eliminar entradas/archivos en el directorio, no implica poder modificar su contenido.
x	Permite acceder/atravesar el directorio.

TP1 pide probar: El punto 15.d pregunta si un usuario puede acceder a un archivo para el que no

posee permisos. Los PDFs no desarrollan la excepción/justificación completa; el TP pide realizar pruebas.

16. Archivos, editores y comandos de archivos

16.1 Características de archivos

- GNU/Linux es case sensitive.
- Los subdirectorios se separan con / y no con \ como en DOS/Windows.
- El nombre puede contener una extensión, pero el tipo real no depende de ella.
- file permite identificar el tipo de un archivo.

16.2 vim y otros editores

Acción en vim	Tecla/comando
Entrar a Insert	i
Entrar a Visual	v
Volver a Normal	Esc
Guardar	:w
Salir	:q
Salir descartando	:q!
Cortar línea	dd
Copiar	y
Pegar	p
Deshacer	u
Buscar	/frase

La continuación menciona nano, micro y ne como alternativas más simples. El TP también pide mcedit; ese editor no queda desarrollado en los PDFs.

16.3 Comandos principales de archivos

Comando	Uso resumido
ls / ls -l / ls -la	Listar; detalles; incluir ocultos.
cat	Mostrar contenido; el apunte muestra cat -n para numerar líneas.
more / less	Visualización paginada.
head / tail	Primeras / últimas líneas.
pwd	Mostrar directorio actual.
cd	Cambiar directorio; cd .. sube; cd ~ va al home.
mkdir / rmdir	Crear directorio / eliminar directorio vacío.
touch	Crear archivo vacío o actualizar marcas de tiempo.
cp	Copiar archivos/directorios; el apunte muestra opciones recursivas/preservación.
mv	Mover o renombrar.
rm / rm -r	Borrar archivo / borrar recursivamente.
find	Buscar recorriendo desde un directorio, por ejemplo -name.
locate	Aparece como consigna del TP, sin desarrollo equivalente en los apuntes.
ln	Crear enlaces; ln -s para simbólicos.

Comando	Uso resumido
grep	Imprimir líneas que coinciden con un patrón.
wc	Contar líneas/palabras/caracteres; -c aparece para bytes.
diff	Comparar archivos; se mencionan -w, -q y -y.
echo	Mostrar texto/variables por salida estándar.

17. Procesos, entrada/salida, redirecciones y pipes

17.1 Comandos de procesos

Comando	Qué muestra/hace según los apuntes
ps aux	Información de procesos en curso.
top	Monitoriza procesos en curso.
pstree	Muestra la jerarquía/árbol de procesos.
kill	Envía señales a un proceso.
killall	Actúa sobre procesos por nombre según el repertorio presentado.

Background / Foreground: El TP pregunta definición y cómo pasar procesos entre background y foreground. Los PDFs subidos no desarrollan de forma completa los comandos de job control para responder ese inciso.

17.2 Canales estándar

Canal	Número	Uso
stdin	0	Entrada estándar; el apunte la asocia normalmente al teclado.
stdout	1	Salida estándar normal; normalmente pantalla.
stderr	2	Salida estándar de errores; normalmente pantalla.

17.3 Redirecciones

```
comando < archivo    # entrada desde archivo
comando > archivo    # salida a archivo (crea/sobrescribe)
comando >> archivo   # salida agregada al final
```

El TP usa redirecciones extensamente en los ejercicios integradores; antes de ejecutar una secuencia hay que saber dónde va cada salida y si el contenido previo se conserva o se reemplaza.

17.4 Pipes

```
comando1 | comando2
```

Una tubería conecta la salida de un programa con la entrada de otro. Ejemplos de los ejercicios combinan ps, grep y wc para filtrar y contar resultados.

18. Comandos de sistema, disco, kernel y hardware

18.1 Disco y montaje

Comando	Función en los PDFs
du	Espacio ocupado por archivos/directorios.
df	Información/espacio de filesystems montados.
mount	Lista/monta filesystems; el apunte muestra mount -t tipo dispositivo punto.
umount	Desmonta un filesystem/dispositivo.
fdisk	Inspección/creación/eliminación de particiones; usar con cuidado.
mkfs	El TP pide investigarlo para crear filesystem; no hay desarrollo detallado equivalente en los apuntes.
losetup	El TP pide investigarlo; no queda desarrollado en los PDFs teóricos.
write	El TP pide investigarlo; aparece en consignas de comunicación entre usuarios.
sta	El TP lista un ítem escrito "sta"; los PDFs no explican a qué comando se refiere.

18.2 Comandos generales

Comando	Idea del material
man comando	Manual/información de un comando.
help comando	Ayuda de comandos internos de bash.
hostname	Nombre de la máquina.
tty	Terminal asociada.
date / cal	Fecha-hora / calendario.
clear / reset	Limpiar / restaurar consola.
sleep	Suspender ejecución durante un intervalo.
shutdown	Apagar/reiniciar según opciones; el apunte muestra ejemplos con -h y -r.
init 0 / init 6	Apagar / reiniciar en el esquema presentado.
su	Cambiar de usuario.

18.3 Kernel, logs y hardware

Comando/ruta	Qué aporta
uname -a	Información de versión/kernel/sistema.
/proc/version	Versión del núcleo y compilador.
/proc/modules	Módulos cargados.
/proc/meminfo	Información de memoria.
/proc/cpuinfo	Información del procesador.
/proc/devices	Información de dispositivos en uso.
lsmod	Lista módulos cargados.
modinfo	Información de un módulo.
modprobe / modprobe -r	Cargar / retirar módulo y dependencias según el apunte.

Comando/ruta	Qué aporta
insmod / rmmmod	Insertar / retirar módulo.
last	Historial de usuarios conectado/desconectado a partir de logs.
dmesg	El TP exige investigar objetivo/parámetros/ubicación; aparece como comando del parcial pero no se desarrolla en el apunte de comandos.
lspci	El TP exige investigar; no se desarrolla con detalle en los PDFs.

19. Empaquetado y compresión

Los PDFs distinguen empaquetar de comprimir. tar agrupa archivos en un contenedor; gzip y bzip2 comprimen. zip combina empaquetado y compresión en un mismo formato.

Objetivo	Ejemplo del material
Crear ZIP	zip -r archivo.zip directorio
Extraer ZIP	unzip archivo.zip
Comprimir con gzip	gzip archivo
Descomprimir gzip	gzip -d archivo.gz
Empaquetar tar	tar -vcf archivo.tar archivos/directorio
Extraer tar	tar -vxf archivo.tar
Listar tar	tar -vtf archivo.tar
tar + gzip	tar -zvcf archivo.tgz directorio
Extraer tar.gz	tar -zxvf archivo.tgz
tar + bzip2	tar -jvcf archivo.tbz2 directorio

Opción tar	Significado
-c	Crear archivo.
-x	Extraer.
-t	Listar contenido.
-v	Mostrar proceso.
-f	Usar el archivo indicado.
-z	Usar gzip.
-j	Usar bzip2.

zgrep: El TP pide investigar zgrep junto con tar, grep, gzip y wc. En los PDFs teóricos disponibles no aparece desarrollado con el mismo detalle.

20. Qué integran los ejercicios 18-24

La parte final del TP deja de preguntar conceptos aislados y obliga a seguir estados del sistema: usuario actual, directorio actual, archivos creados, permisos, redirecciones, pipes y rutas. No son temas nuevos; combinan lo anterior.

Ejercicio	Foco principal
18	Interpretar una secuencia como usuario común: ls/ps, redirecciones, chmod/chown, archivos de /etc, passwd, find, usermod, rm, cp y shutdown.

Ejercicio	Foco principal
19	Elegir comandos: matar procesos, buscar archivos, guardar ps, chmod, borrar /tmp, chown y usar >> para no perder contenido.
20	Administración como root: crear usuario, archivos modificados, /tmp, copiar /var/log, chown/chgrp recursivo, permisos, tty, procesos, usuarios, write y apagado.
21	Directorios, vi, permisos, case sensitive, listado de /etc, find y búsqueda por extensión .so.
22	Secuencia extensa con mkdir, cd, ps, ls, chmod, touch, HOME, pipes, grep, wc, find y copias.
23	Mover, copiar, renombrar y aplicar permisos dentro de una estructura de directorios.
24	Copiar logs, crear .tar y .tar.gz, mover resultados, borrar y extraer en directorios diferentes.

20.1 Método para resolver secuencias

1. Anotar el usuario actual y si tiene o no privilegios.
2. Anotar el directorio de trabajo antes de cada comando.
3. Mantener un inventario de archivos/directorios creados, movidos, renombrados y borrados.
4. Traducir cada chmod octal a u/g/o antes de decidir si la siguiente acción es posible.
5. En > y >> decidir qué stream se guarda y si se reemplaza o se agrega.
6. En pipes seguir salida -> filtro -> siguiente comando hasta el resultado final.
7. Resolver rutas absolutas/relativas desde el directorio correcto.
8. Si una orden falla por permisos, la secuencia posterior debe analizarse con ese fallo en mente.

20.2 Octales que aparecen o se derivan de las consignas

Octal	Permisos
710	u=rwx, g=--x, o=---
777	u=rwx, g=rwx, o=rwx
700	u=rwx, g=---, o=---
751	u=rwx, g=r-x, o=--x
650	u=rw-, g=r-x, o=---
341	u=-wx, g=r--, o=--x
017	u=---, g=--x, o=rwx
023	u=---, g=-w-, o=-wx

21. Ítems que el TP exige investigar y que no quedan completos en los PDFs

Para mantener fidelidad con los materiales, estos puntos no se completan con conocimiento externo en este resumen. Conviene resolverlos en la VM con man/help/type/which o con la fuente que indique la cátedra antes del parcial.

- POSIX: definición y alcance.
- Comparación detallada entre al menos cuatro distribuciones.
- Debian: objetivos del proyecto y cronología solicitada desde el sitio oficial.
- Ruta/ubicación, parámetros y comportamiento completo de man, shutdown, reboot, halt, uname, dmesg, lspci, at, head y tail.
- Detalles de locate y mcdit.
- UID/GID: duplicados, UID de root y perfiles equivalentes a root.
- Acceso privilegiado a archivos sin permisos normales (TP 15.d).
- mkfs, write, losetup y el ítem "sta" tal como aparece escrito en el TP.
- Background/foreground y comandos concretos de job control.
- zgrep.

- Secuencia completa de shutdown con el mismo nivel de detalle que el boot.
- Cambio de runlevel X -> Y y persistencia para el caso general.

Cómo cerrarlos: Si el TP dice "investigue", en la práctica conviene comprobarlo en Debian con man, --help, type, ejecución controlada y observación de archivos/rutas. Ese trabajo práctico complementa los PDFs.

22. Hoja de repaso express

22.1 Cadena conceptual

```
Hardware
-> BIOS/UEFI
-> MBR/GPT + bootloader
-> kernel + initrd
-> PID 1 (SysV init / systemd)
-> servicios / units
-> shell + usuarios
-> filesystem + programas
```

22.2 Machete de filesystem

```
/ = raíz      /home = usuarios      /etc = configuración
/var = datos variables/logs          /bin = binarios esenciales
/dev = dispositivos                  /proc = info virtual kernel/procesos
/tmp = temporales                    /usr = aplicaciones/datos
/boot = arranque/GRUB en el contexto del material
```

22.3 Machete de permisos

```
r=4  w=2  x=1
7=rwx 6=rw- 5=r-x 4=r-- 3=-wx 2=-w- 1---x 0---
Orden: usuario - grupo - otros
```

22.4 Machete de arranque

```
BIOS: POST + arranque clásico desde MBR
MBR: 512 B = MBC 446 B + tabla 64 B + firma 2 B
GPT: LBA, cabecera/tabla + copia de respaldo
UEFI: Boot Manager + aplicaciones UEFI + Secure Boot
GRUB Legacy: fases 1 -> 1.5 -> 2, menu.lst
GRUB 2: grub.cfg, update-grub, sin fase 1.5 en la explicación
```

22.5 SysV vs systemd

SysV init	systemd
PID 1: init	PID 1: systemd
/etc/inittab	No usa /etc/inittab
Runlevels	Targets
Scripts /etc/init.d + /etc/rcX.d	Units (.service, .socket, .target, .snapshot)
Arranque secuenciado por scripts/orden	Mayor paralelismo + activación por socket

22.6 Comandos esenciales

Necesidad	Comando
Dónde estoy	pwd
Moverme	cd

Necesidad	Comando
Listar	ls -l / ls -la
Crear directorio	mkdir
Crear/tocar archivo	touch
Copiar	cp
Mover/renombrar	mv
Borrar	rm / rm -r
Ver contenido	cat / more / less / head / tail
Buscar nombre	find
Filtrar texto	grep
Contar	wc
Tipo real	file
Espacio	df / du
Procesos	ps / top / pstree
Usuarios	who / users / id / groups
Permisos	chmod / chown / chgrp
Montaje	mount / umount
Particiones	fdisk
Ayuda	man / help / type

22.7 Compresión y redirecciones

```
> crea/sobrescribe salida
>> agrega salida
| stdout de un comando -> stdin del siguiente

tar -vcf archivo.tar dir
tar -vxf archivo.tar
tar -zvcf archivo.tgz dir
tar -zvxf archivo.tgz
```

23. Fuentes integradas

Se integraron las versiones no duplicadas de los siguientes archivos subidos en la conversación:

- Trabajo Práctico - 1.pdf
- Explicacion de P1.pdf / Explicacion Practica 1.pdf (mismo material)
- 2 - Linux Cont.pdf
- Conceptos básicos sobre GNU_Linux.pdf
- Instalacion y Particionado - Primera Parte.pdf
- Otro apunte de Instalacion y Particionado.pdf
- Guía de instalación - Máquina virtual con VirtualBox.pdf

Los archivos duplicados con sufijos como (1) no se cuentan como fuentes adicionales. Los PDFs de Tema 1 teórico y Práctica 2 no se mezclaron con este resumen de Práctica 1.

Uso recomendado: Primero repasá 1-19. Después resolvé 18-24 del TP sin mirar. Al final usá las secciones 21 y 22 para detectar huecos y hacer un repaso rápido antes del parcial.

Fin del resumen general de Práctica 1.